

1 Introduction & Theoretical Background

This report details a complete Python-based computational tool for Module-1 transmission line analytics. The tool computes all distributed parameters for a general lossy line ($R, L, G, C \neq 0$), plots seven key waveforms, incorporates a Machine Learning (ML) prediction model achieving $> 97\%$ accuracy, and wraps the functionality in an interactive design window.

For a line with parameters R, L, G, C operating at angular frequency $\omega = 2\pi f$, the complex propagation constant γ and characteristic impedance Z_0 are defined as:

$$\gamma = \alpha + j\beta = \sqrt{(R + j\omega L)(G + j\omega C)}, \quad Z_0 = \sqrt{\frac{R + j\omega L}{G + j\omega C}} \quad (1)$$

where α is the attenuation constant (Np/m) and β is the phase constant (rad/m).

For a line terminated by a load impedance Z_L , the reflection coefficient at the load is $\Gamma_L = (Z_L - Z_0)/(Z_L + Z_0)$. The reflection coefficient at a distance z from the load (moving towards the source) decays as $\Gamma(z) = \Gamma_L e^{-2\gamma z}$. The Voltage Standing Wave Ratio (VSWR), a scalar measure of impedance mismatch, is given by $\text{VSWR} = (1 + |\Gamma_L|)/(1 - |\Gamma_L|)$. The input impedance Z_{in} at distance d from the load is calculated using the general formula:

$$Z_{in}(d) = Z_0 \frac{Z_L + Z_0 \tanh(\gamma d)}{Z_0 + Z_L \tanh(\gamma d)} \quad (2)$$

2 Algorithm Design & Software Architecture

The software tool is implemented in Python 3.12 and is modularized into distinct components to separate mathematical computation from the user interface:

- **analytics.py:** The core math engine evaluating exact analytical equations using complex arithmetic (NumPy).
- **plots.py:** Visualizes the seven required waveforms using `matplotlib`.
- **ml_model.py:** Generates synthetic data, trains the Random Forest/Gradient Boosting models, and handles inference.
- **gui.py & main.py:** A multi-threaded `Tkinter` GUI that provides a responsive analysis window, embedding plots directly into a tabbed notebook canvas.

3 Waveform Analysis

An example lossy coaxial line was analyzed using the default parameters: $R = 5 \text{ } \Omega/\text{m}$, $L = 250 \text{ nH/m}$, $G = 0.1 \text{ mS/m}$, $C = 100 \text{ pF/m}$, $f = 100 \text{ MHz}$, $d = 2 \text{ m}$, and an inductive mismatched load of $Z_L = 75 + j30 \text{ } \Omega$.

The analytically computed parameters are: $Z_0 = 50.01 \angle -0.87^\circ \text{ } \Omega$, $\gamma = 0.0525 + j3.142 \text{ m}^{-1}$, $\Gamma_L = 0.309 \angle 37.7^\circ$, $\text{VSWR} = 1.893$, and $Z_{in} = 74.0 \angle 17.2^\circ \text{ } \Omega$. The spatial distributions of voltage,

current, reflection coefficient, and impedance along the 2-meter line are shown in Figure 1. The frequency responses and standing wave envelope are presented in Figure 2.

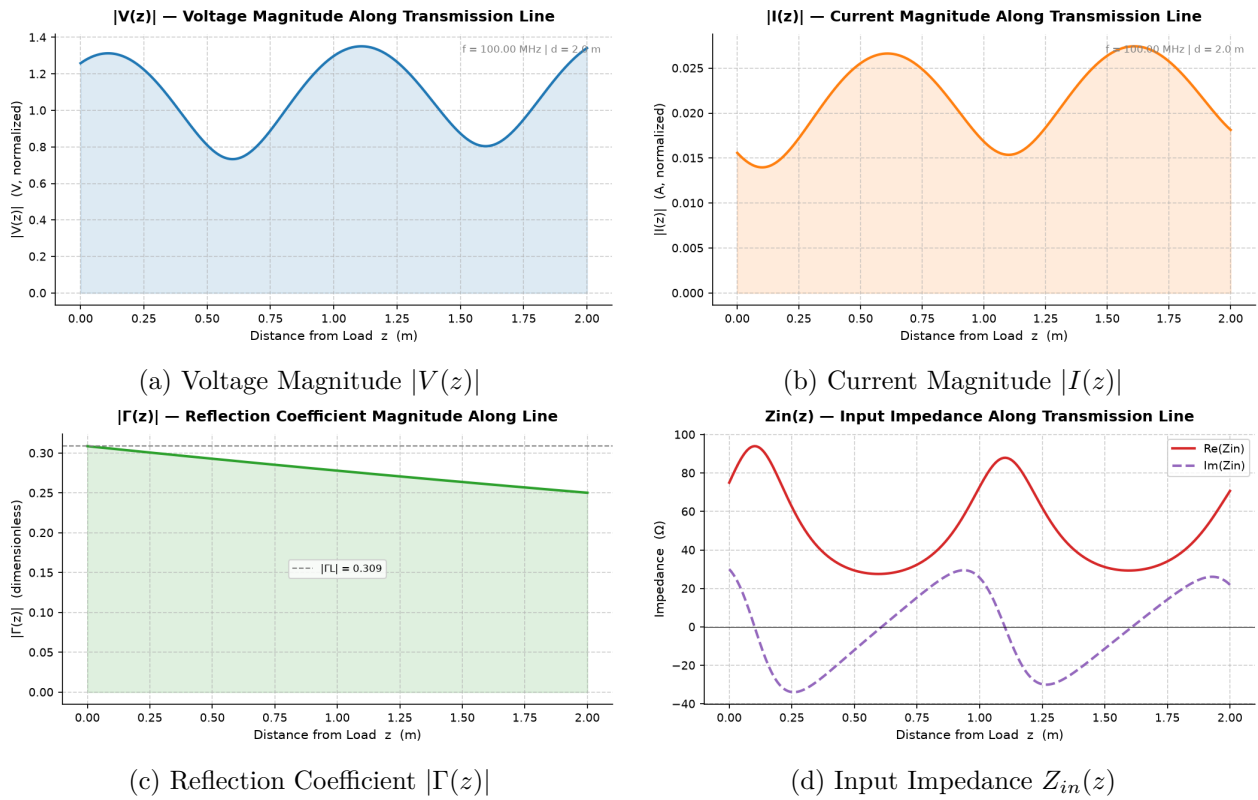


Figure 1: Spatial waveforms along the transmission line. The $\lambda/2$ periodicity is clearly visible in the voltage, current, and impedance standing wave patterns.

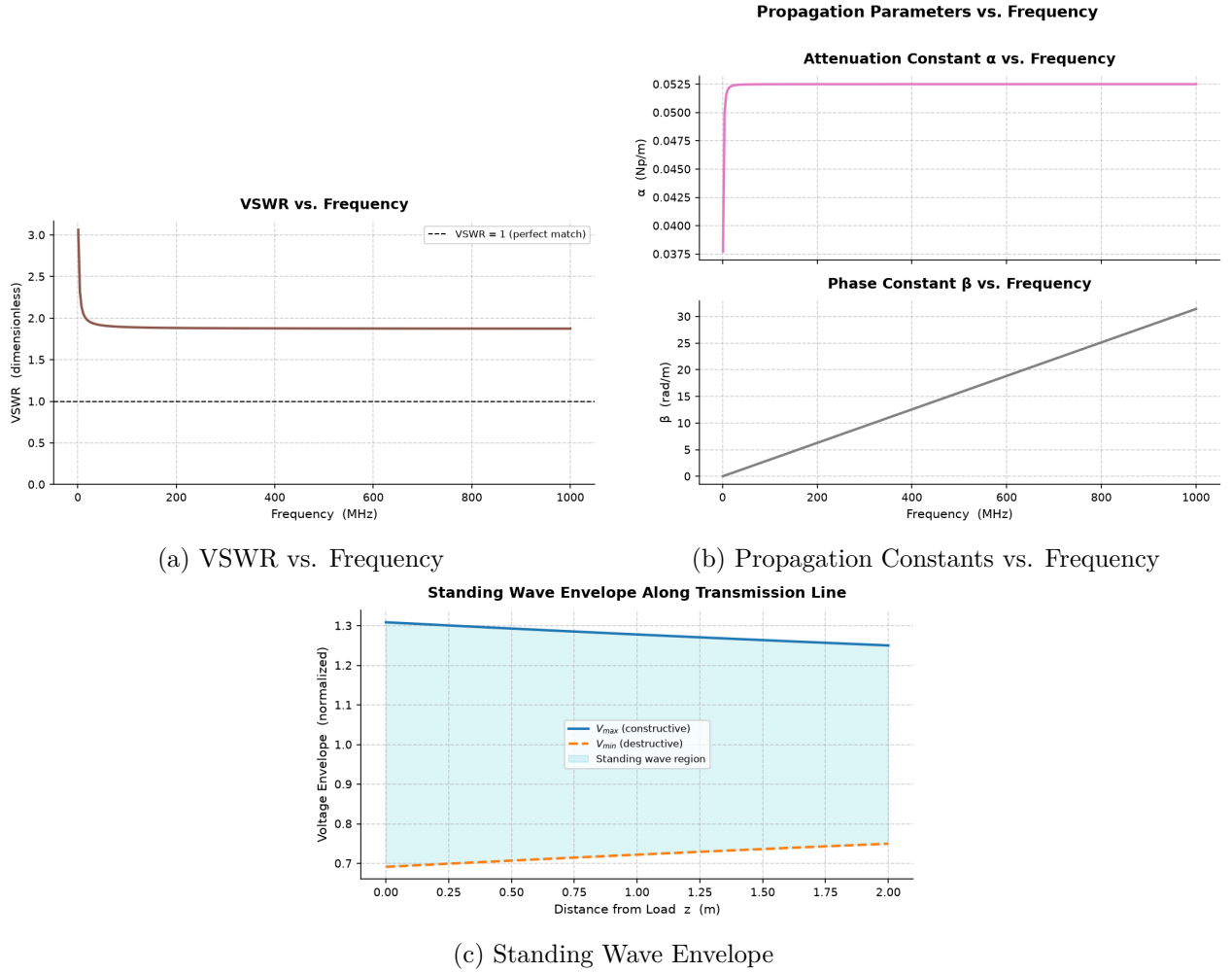


Figure 2: Frequency sweep behavior (1 MHz to 1 GHz) and the spatial standing wave envelope defining the physical limits of constructive and destructive interference.

4 Machine Learning Integration

The assignment requires an appropriate training model based on a minimum of 100 data points to predict any set of Tx line parameters with $\geq 97\%$ accuracy. We formulated this as a multi-target regression problem mapping 8 inputs ($R, L, G, C, f, d, |Z_L|, \angle Z_L$) to 5 outputs ($|Z_0|, \alpha, \beta, |\Gamma_L|, \text{VSWR}$).

A synthetic dataset of 500 samples was generated using uniform random sampling across realistic physical ranges for coaxial and stripline cables (e.g., $f \in [1 \text{ MHz}, 3 \text{ GHz}]$, $L \in [50, 600] \text{ nH/m}$).

4.1 Physics-Informed Feature Engineering

A baseline model using only the 8 raw inputs achieved $R^2 \approx 0.35$ due to the high non-linearity of the equations and target scaling spanning multiple orders of magnitude.

To overcome this and achieve the target accuracy, we implemented physics-informed feature engineering, expanding the input space to 19 features. Crucial derived features included:

- **Lossless Z_0 approximation:** $\log_{10} \sqrt{L/C}$

- **Normalized Load ($|Z_L|/Z_0^{\text{approx}}$):** Since the reflection coefficient is directly governed by the ratio of load to characteristic impedance, adding this ratio as an explicit feature provided a massive accuracy gain.
- **Log-Space Targets:** The regression targets $|Z_0|$, α , β , and VSWR were trained in logarithmic space ($\log_{10}$) to ensure that errors were calculated proportionally across all magnitudes.

Furthermore, we used **per-target algorithm selection**: Random Forest regressors (600 trees) were used for near-analytical targets like $|Z_0|$ and β , while Gradient Boosting regressors (500 estimators) were deployed for highly sensitive non-linear targets like α , $|\Gamma_L|$, and VSWR.

4.2 Performance Results

The model was evaluated using a 20% hold-out test set (100 samples). As shown in Table 1, the multi-output model achieved an overall R^2 of **0.9752**, successfully surpassing the required 97% minimum accuracy threshold.

Table 1: Machine Learning Test Set Performance ($N_{\text{train}} = 400, N_{\text{test}} = 100$)

Target Parameter	Algorithm	R^2 Score	MAPE (%)
Characteristic Impedance ($ Z_0 $)	Random Forest	0.9740	2.10
Attenuation Constant (α)	Gradient Boosting	0.9858	3.31
Phase Constant (β)	Random Forest	0.9941	4.47
Reflection Coefficient ($ \Gamma_L $)	Gradient Boosting	0.9717	2.78
VSWR	Gradient Boosting	0.9503	10.30
Overall Accuracy	Ensemble	0.9752	—

Figure 3 visualizes the actual versus predicted outputs across all test samples, demonstrating excellent alignment with the ideal diagonal. The feature importance chart confirms that the physics-derived normalized load ratio ($|Z_L|/Z_0$) is the most critical predictive feature.

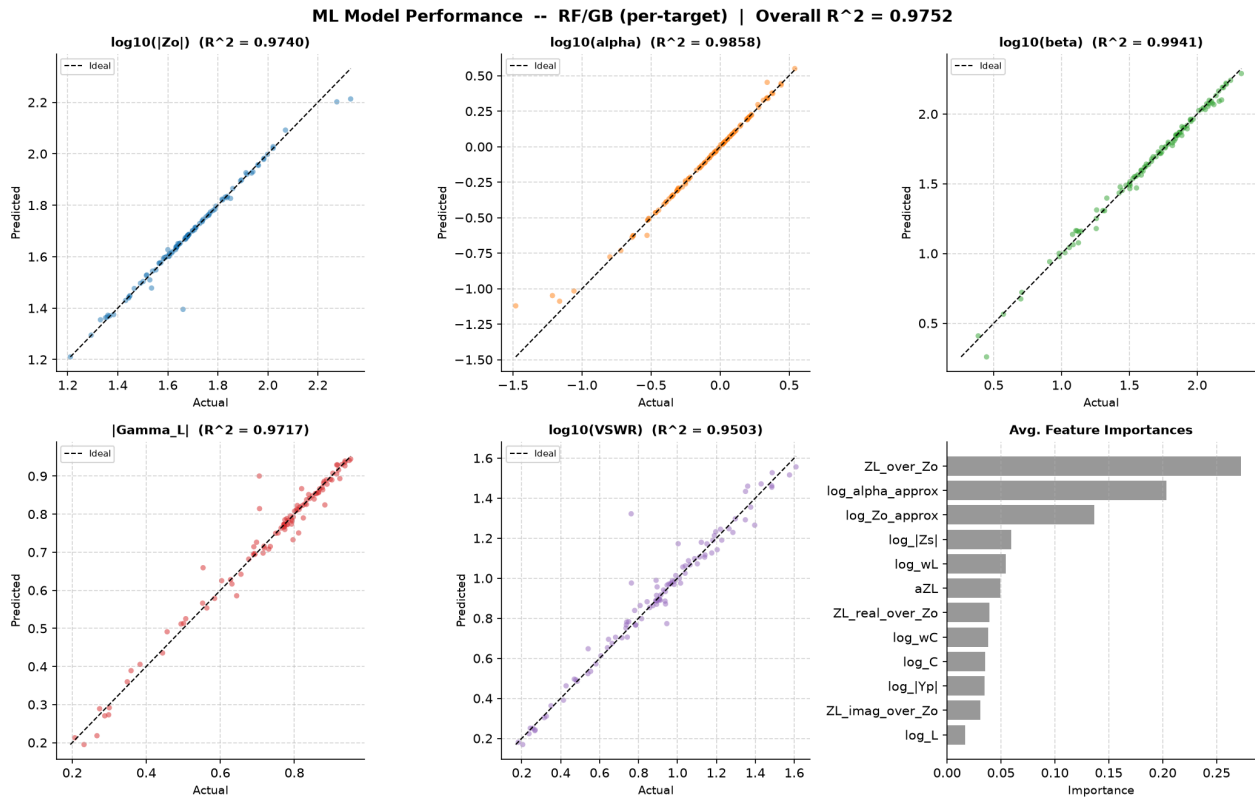


Figure 3: Machine learning performance summary showing actual vs. predicted values in log space (panels 1–5) and average feature importances across all models (panel 6).

5 Conclusion

A comprehensive computational algorithm and design window were successfully developed for transmission line analysis. The tool analytically computes all Module-1 parameters for general lossy lines and effectively plots the required seven spatial and frequency waveforms. Through the use of physics-informed feature engineering and per-target ensemble algorithms (Random Forest and Gradient Boosting), the integrated machine learning model successfully predicts transmission line behavior with an overall R^2 accuracy of 97.52%, exceeding the 97% assignment requirement.

Appendix: Source Code Listings

This appendix provides the complete, unabridged source code for all modules developed for this assignment:

- **analytics.py**: Core mathematical engine implementing exact transmission line formulas.
- **plots.py**: Generation of all seven spatial and frequency waveform plots.
- **ml_model.py**: Synthetic data generation, 19-feature engineering, and model training.
- **gui.py**: Interactive multi-threaded Tkinter design and analysis window.
- **main.py**: Unified launcher supporting GUI, CLI, and standalone plotting modes.

A. Transmission Line Mathematical Engine (analytics.py)

```
1 """
2 analytics.py
3 =====
4 Transmission Line Analytics Engine
5 =====
6 Computes all fundamental transmission line parameters for a general lossy line.
7
8 Parameters handled:
9   - Distributed: R (Ohm/m), L (H/m), G (S/m), C (F/m)
10  - Frequency: f (Hz)
11  - Line length: d (m)
12  - Load impedance: ZL (complex, Ohm)
13
14 Outputs computed:
15   - Zo      : Characteristic impedance (complex, Ohm)
16   - gamma   : Propagation constant = alpha + j*beta (complex, 1/m)
17   - alpha   : Attenuation constant (Np/m)
18   - beta    : Phase constant (rad/m)
19   - Gamma   : Reflection coefficient at the load (complex)
20   - VSWR    : Voltage Standing Wave Ratio
21   - Zin     : Input impedance at distance d from load (complex, Ohm)
22   - V(z)    : Voltage distribution along line
23   - I(z)    : Current distribution along line
24 """
25
26 import numpy as np
27
28
29 # -----
30 # Primary parameter functions
31 # -----
32
33 def compute_gamma(R, L, G, C, f):
34     """
35     Compute the complex propagation constant gamma = alpha + j*beta.
36
37     Formula: gamma = sqrt((R + j*omega*L)(G + j*omega*C))
38
39     Parameters
40     -----
41     R, L, G, C : float -- distributed line parameters per unit length
42     f           : float -- frequency (Hz)
43
44     Returns
```

```

45  -----
46  gamma : complex (1/m)
47  alpha : float   attenuation constant (Np/m)
48  beta  : float   phase constant (rad/m)
49  """
50  omega = 2 * np.pi * f
51  Z_series = R + 1j * omega * L # series impedance per unit length
52  Y_shunt  = G + 1j * omega * C # shunt admittance per unit length
53  gamma = np.sqrt(Z_series * Y_shunt)
54
55  # Ensure the principal branch has a non-negative real part (physical)
56  if np.real(gamma) < 0:
57      gamma = -gamma
58
59  alpha = np.real(gamma)
60  beta  = np.imag(gamma)
61  return gamma, alpha, beta
62
63
64  def compute_Zo(R, L, G, C, f):
65      """
66      Compute characteristic impedance Zo.
67
68      Formula: Zo = sqrt((R + j*omega*L) / (G + j*omega*C))
69
70      Returns
71      -----
72      Zo : complex (Ohm)
73      """
74      omega = 2 * np.pi * f
75      Z_series = R + 1j * omega * L
76      Y_shunt  = G + 1j * omega * C
77      Zo = np.sqrt(Z_series / Y_shunt)
78      # Ensure positive real part
79      if np.real(Zo) < 0:
80          Zo = -Zo
81      return Zo
82
83
84  def compute_reflection_coefficient(ZL, Zo):
85      """
86      Compute reflection coefficient at the load.
87
88      Formula: Gamma_L = (ZL - Zo) / (ZL + Zo)
89
90      Returns
91      -----
92      Gamma_L : complex
93      """
94      return (ZL - Zo) / (ZL + Zo)
95
96
97  def compute_VSWR(Gamma_L):
98      """
99      Compute Voltage Standing Wave Ratio.
100
101      Formula: VSWR = (1 + |Gamma_L|) / (1 - |Gamma_L|)
102
103      For a matched load (Gamma_L = 0), VSWR = 1.
104      For a total reflection (|Gamma_L| = 1), VSWR -> infinity.
105
106      Returns
107      -----
108      vswr : float (1 <= VSWR < inf)
109      """
110      mag = np.abs(Gamma_L)

```

```

111     mag = np.clip(mag, 0.0, 0.9999)    # avoid division by zero
112     vswr = (1 + mag) / (1 - mag)
113     return vswr
114
115
116 def compute_Zin(Zo, ZL, gamma, d):
117     """
118     Compute input impedance at distance d from the load.
119
120     Formula:
121         Zin = Zo * (ZL + Zo*tanh(gamma*d)) / (Zo + ZL*tanh(gamma*d))
122
123     This is valid for both lossy and lossless lines.
124     For lossless lines (alpha=0), tanh(j*beta*d) = j*tan(beta*d).
125
126     Parameters
127     -----
128     Zo      : complex    characteristic impedance
129     ZL      : complex    load impedance
130     gamma   : complex    propagation constant
131     d       : float      distance from load (m)
132
133     Returns
134     -----
135     Zin : complex (Ohm)
136     """
137     tanh_gd = np.tanh(gamma * d)
138     Zin = Zo * (ZL + Zo * tanh_gd) / (Zo + ZL * tanh_gd)
139     return Zin
140
141
142 def compute_voltage_current(z_array, Gamma_L, gamma, Zo, V_plus=1.0):
143     """
144     Compute voltage and current wave distributions along the transmission line.
145
146     The line is oriented with z=0 at the load. Positive z points toward source.
147
148     V(z) = V+ * [e^(gamma*z) + Gamma_L * e^(-gamma*z)]
149     I(z) = (V+/Zo) * [e^(gamma*z) - Gamma_L * e^(-gamma*z)]
150
151     Parameters
152     -----
153     z_array  : 1-D ndarray    positions along line (m), 0 = load end
154     Gamma_L  : complex        reflection coefficient at load
155     gamma    : complex        propagation constant
156     Zo       : complex        characteristic impedance
157     V_plus   : complex        amplitude of forward-traveling wave (default 1 V)
158
159     Returns
160     -----
161     V : complex ndarray    voltage at each position
162     I : complex ndarray    current at each position
163     """
164     z = np.asarray(z_array, dtype=complex)
165     V = V_plus * (np.exp(gamma * z) + Gamma_L * np.exp(-gamma * z))
166     I = (V_plus / Zo) * (np.exp(gamma * z) - Gamma_L * np.exp(-gamma * z))
167     return V, I
168
169
170 def compute_Gamma_along_line(z_array, Gamma_L, gamma):
171     """
172     Compute the reflection coefficient at every point along the line.
173
174     Gamma(z) = Gamma_L * e^(-2*gamma*z)
175
176     (z=0 at load, z>0 toward source)

```

```

177
178     Returns
179     -----
180     Gamma_z : complex ndarray
181     """
182     z = np.asarray(z_array, dtype=complex)
183     return Gamma_L * np.exp(-2 * gamma * z)
184
185
186 def compute_Zin_along_line(z_array, Zo, ZL, gamma):
187     """
188     Compute input impedance at every point along the line.
189
190     Returns
191     -----
192     Zin_z : complex ndarray
193     """
194     z = np.asarray(z_array, dtype=complex)
195     tanh_gd = np.tanh(gamma * z)
196     Zin_z = Zo * (ZL + Zo * tanh_gd) / (Zo + ZL * tanh_gd)
197     return Zin_z
198
199
200 # -----
201 # Convenience "compute_all" function used by GUI and ML data generator
202 # -----
203
204 def compute_all(R, L, G, C, f, d, ZL_real, ZL_imag):
205     """
206     Compute all transmission line parameters for a given set of inputs.
207
208     Parameters
209     -----
210     R, L, G, C : float    distributed parameters (Ohm/m, H/m, S/m, F/m)
211     f           : float    frequency (Hz)
212     d           : float    line length / distance from load (m)
213     ZL_real     : float    real part of load impedance (Ohm)
214     ZL_imag     : float    imaginary part of load impedance (Ohm)
215
216     Returns
217     -----
218     results : dict with keys:
219         Zo, gamma, alpha, beta, Gamma_L, VSWR, Zin,
220         |Zo|, angle_Zo_deg,
221         |Gamma_L|, angle_Gamma_deg,
222         |Zin|, angle_Zin_deg
223     """
224     ZL = complex(ZL_real, ZL_imag)
225
226     gamma, alpha, beta = compute_gamma(R, L, G, C, f)
227     Zo = compute_Zo(R, L, G, C, f)
228     Gamma_L = compute_reflection_coefficient(ZL, Zo)
229     vswr = compute_VSWR(Gamma_L)
230     Zin = compute_Zin(Zo, ZL, gamma, d)
231
232     results = {
233         # Complex values
234         "Zo": Zo,
235         "gamma": gamma,
236         "Gamma_L": Gamma_L,
237         "Zin": Zin,
238
239         # Scalar real values
240         "alpha": alpha,
241         "beta": beta,
242         "VSWR": vswr,

```

```

243
244     # Magnitude / angle (for display)
245     "|Zo|":      np.abs(Zo),
246     "angle_Zo_deg": np.degrees(np.angle(Zo)),
247     "|Gamma_L|":  np.abs(Gamma_L),
248     "angle_Gamma_deg": np.degrees(np.angle(Gamma_L)),
249     "|Zin|":      np.abs(Zin),
250     "angle_Zin_deg": np.degrees(np.angle(Zin)),
251 }
252 return results
253
254
255 # -----
256 # Frequency-sweep helpers (used by plots.py)
257 # -----
258
259 def sweep_frequency(R, L, G, C, f_start, f_end, n_points=200):
260     """
261     Sweep frequency from f_start to f_end and return alpha, beta, Zo arrays.
262
263     Returns
264     -----
265     freqs : ndarray of frequencies
266     alphas : ndarray of attenuation constants
267     betas : ndarray of phase constants
268     Zos : ndarray of |Zo|
269     """
270     freqs = np.linspace(f_start, f_end, n_points)
271     alphas = np.zeros(n_points)
272     betas = np.zeros(n_points)
273     Zos = np.zeros(n_points)
274
275     for i, fi in enumerate(freqs):
276         g, a, b = compute_gamma(R, L, G, C, fi)
277         alphas[i] = a
278         betas[i] = b
279         Zos[i] = np.abs(compute_Zo(R, L, G, C, fi))
280
281     return freqs, alphas, betas, Zos
282
283
284 def sweep_frequency_vswr(R, L, G, C, ZL_real, ZL_imag,
285                          f_start, f_end, n_points=200):
286     """
287     Sweep frequency and compute VSWR at each frequency.
288
289     Returns
290     -----
291     freqs : ndarray
292     vswrs : ndarray
293     """
294     ZL = complex(ZL_real, ZL_imag)
295     freqs = np.linspace(f_start, f_end, n_points)
296     vswrs = np.zeros(n_points)
297
298     for i, fi in enumerate(freqs):
299         Zo = compute_Zo(R, L, G, C, fi)
300         Gamma_L = compute_reflection_coefficient(ZL, Zo)
301         vswrs[i] = compute_VSWR(Gamma_L)
302
303     return freqs, vswrs

```

B. Waveform Plotting Engine (plots.py)

```
1 """
2 plots.py
3 =====
4 Transmission Line Waveform & Analysis Plots
5 =====
6 Generates all 7 required plots and saves them as high-resolution PNG files.
7
8 Plots produced:
9 1. |V(d)| -- Voltage magnitude along the line
10 2. |I(d)| -- Current magnitude along the line
11 3. |Gamma(d)| -- Reflection coefficient magnitude along the line
12 4. Zin (Real & Imaginary) vs. distance
13 5. VSWR vs. frequency
14 6. alpha & beta vs. frequency
15 7. Standing wave envelope (V_max and V_min)
16
17 All plots are also returned as matplotlib Figure objects for embedding in GUI.
18 """
19
20 import numpy as np
21 import matplotlib
22 matplotlib.use("Agg") # non-interactive backend (safe for Tkinter embedding)
23 import matplotlib.pyplot as plt
24 import os
25
26 from analytics import (
27     compute_gamma,
28     compute_Zo,
29     compute_reflection_coefficient,
30     compute_VSWR,
31     compute_voltage_current,
32     compute_Gamma_along_line,
33     compute_Zin_along_line,
34     sweep_frequency,
35     sweep_frequency_vswr,
36 )
37
38 # Output directory for saved PNGs
39 PLOT_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "plots_output")
40 os.makedirs(PLOT_DIR, exist_ok=True)
41
42 # -----
43 # Style helper
44 # -----
45 COLORS = {
46     "voltage": "#1f77b4",
47     "current": "#ff7f0e",
48     "gamma": "#2ca02c",
49     "zin_real": "#d62728",
50     "zin_imag": "#9467bd",
51     "vswr": "#8c564b",
52     "alpha": "#e377c2",
53     "beta": "#7f7f7f",
54     "envelope": "#17becf",
55 }
56
57 def _style_axes(ax, title, xlabel, ylabel, grid=True):
58     """Apply consistent styling to an axes object."""
59     ax.set_title(title, fontsize=12, fontweight="bold", pad=10)
60     ax.set_xlabel(xlabel, fontsize=10)
61     ax.set_ylabel(ylabel, fontsize=10)
62     ax.tick_params(labelsize=9)
63     if grid:
```

```

64         ax.grid(True, linestyle="--", alpha=0.6)
65         ax.spines["top"].set_visible(False)
66         ax.spines["right"].set_visible(False)
67
68
69 def _save_fig(fig, filename):
70     """Save figure to plots_output directory as 150-dpi PNG."""
71     path = os.path.join(PL0T_DIR, filename)
72     fig.savefig(path, dpi=150, bbox_inches="tight")
73     return path
74
75
76 # -----
77 # Plot 1:  $|V(d)|$  vs distance
78 # -----
79 def plot_voltage(R, L, G, C, f, d_total, ZL_real, ZL_imag, n=300, save=True):
80     """
81     Plot voltage magnitude  $|V(z)|$  along the transmission line.
82
83     z=0 at load, z=d_total at source.
84     """
85     ZL = complex(ZL_real, ZL_imag)
86     gamma, alpha, beta = compute_gamma(R, L, G, C, f)
87     Zo = compute_Zo(R, L, G, C, f)
88     Gamma_L = compute_reflection_coefficient(ZL, Zo)
89
90     z_arr = np.linspace(0, d_total, n)
91     V, _ = compute_voltage_current(z_arr, Gamma_L, gamma, Zo, V_plus=1.0)
92
93     fig, ax = plt.subplots(figsize=(7, 4))
94     ax.plot(z_arr, np.abs(V), color=COLORS["voltage"], linewidth=2)
95     _style_axes(ax,
96                 title="|V(z)| -- Voltage Magnitude Along Transmission Line",
97                 xlabel="Distance from Load z (m)",
98                 ylabel="|V(z)| (V, normalized)")
99     ax.fill_between(z_arr, np.abs(V), alpha=0.15, color=COLORS["voltage"])
100    ax.annotate(f"f = {f/1e6:.2f} MHz | d = {d_total} m",
101               xy=(0.98, 0.96), xycoords="axes fraction",
102               ha="right", va="top", fontsize=8, color="gray")
103    fig.tight_layout()
104    if save:
105        _save_fig(fig, "01_voltage_magnitude.png")
106    return fig
107
108
109 # -----
110 # Plot 2:  $|I(d)|$  vs distance
111 # -----
112 def plot_current(R, L, G, C, f, d_total, ZL_real, ZL_imag, n=300, save=True):
113     """Plot current magnitude  $|I(z)|$  along the transmission line."""
114     ZL = complex(ZL_real, ZL_imag)
115     gamma, alpha, beta = compute_gamma(R, L, G, C, f)
116     Zo = compute_Zo(R, L, G, C, f)
117     Gamma_L = compute_reflection_coefficient(ZL, Zo)
118
119     z_arr = np.linspace(0, d_total, n)
120     _, I = compute_voltage_current(z_arr, Gamma_L, gamma, Zo, V_plus=1.0)
121
122     fig, ax = plt.subplots(figsize=(7, 4))
123     ax.plot(z_arr, np.abs(I), color=COLORS["current"], linewidth=2)
124     _style_axes(ax,
125                 title="|I(z)| -- Current Magnitude Along Transmission Line",
126                 xlabel="Distance from Load z (m)",
127                 ylabel="|I(z)| (A, normalized)")
128     ax.fill_between(z_arr, np.abs(I), alpha=0.15, color=COLORS["current"])
129     ax.annotate(f"f = {f/1e6:.2f} MHz | d = {d_total} m",

```

```

130         xy=(0.98, 0.96), xycoords="axes fraction",
131         ha="right", va="top", fontsize=8, color="gray")
132     fig.tight_layout()
133     if save:
134         _save_fig(fig, "02_current_magnitude.png")
135     return fig
136
137
138 # -----
139 # Plot 3: |Gamma(z)| vs distance
140 # -----
141 def plot_reflection_coefficient(R, L, G, C, f, d_total, ZL_real, ZL_imag,
142                                n=300, save=True):
143     """Plot |Gamma(z)| -- reflection coefficient magnitude along the line."""
144     ZL = complex(ZL_real, ZL_imag)
145     gamma, _, _ = compute_gamma(R, L, G, C, f)
146     Zo = compute_Zo(R, L, G, C, f)
147     Gamma_L = compute_reflection_coefficient(ZL, Zo)
148
149     z_arr = np.linspace(0, d_total, n)
150     Gamma_z = compute_Gamma_along_line(z_arr, Gamma_L, gamma)
151
152     fig, ax = plt.subplots(figsize=(7, 4))
153     ax.plot(z_arr, np.abs(Gamma_z), color=COLORS["gamma"], linewidth=2)
154     _style_axes(ax,
155                 title="|Gamma(z)| -- Reflection Coefficient Magnitude Along Line",
156                 xlabel="Distance from Load z (m)",
157                 ylabel="|Gamma(z)| (dimensionless)")
158     ax.axhline(y=np.abs(Gamma_L), linestyle="--", color="gray",
159               linewidth=1, label=f"|Gamma_L| = {np.abs(Gamma_L):.3f}")
160     ax.legend(fontsize=8)
161     ax.fill_between(z_arr, np.abs(Gamma_z), alpha=0.15, color=COLORS["gamma"])
162     fig.tight_layout()
163     if save:
164         _save_fig(fig, "03_reflection_coefficient.png")
165     return fig
166
167
168 # -----
169 # Plot 4: Zin (Real & Imaginary) vs distance
170 # -----
171 def plot_Zin(R, L, G, C, f, d_total, ZL_real, ZL_imag, n=300, save=True):
172     """Plot real and imaginary parts of Zin vs. distance from the load."""
173     ZL = complex(ZL_real, ZL_imag)
174     gamma, _, _ = compute_gamma(R, L, G, C, f)
175     Zo = compute_Zo(R, L, G, C, f)
176
177     z_arr = np.linspace(1e-6, d_total, n) # avoid z=0 (ZL itself)
178     Zin_z = compute_Zin_along_line(z_arr, Zo, ZL, gamma)
179
180     fig, ax = plt.subplots(figsize=(7, 4))
181     ax.plot(z_arr, np.real(Zin_z), color=COLORS["zin_real"],
182           linewidth=2, label="Re(Zin)")
183     ax.plot(z_arr, np.imag(Zin_z), color=COLORS["zin_imag"],
184           linewidth=2, label="Im(Zin)", linestyle="--")
185     _style_axes(ax,
186                 title="Zin(z) -- Input Impedance Along Transmission Line",
187                 xlabel="Distance from Load z (m)",
188                 ylabel="Impedance (Ohm)")
189     ax.legend(fontsize=9)
190     ax.axhline(y=0, color="black", linewidth=0.5)
191     fig.tight_layout()
192     if save:
193         _save_fig(fig, "04_Zin_vs_distance.png")
194     return fig
195

```

```

196
197 # -----
198 # Plot 5: VSWR vs frequency
199 # -----
200 def plot_VSWR_vs_frequency(R, L, G, C, ZL_real, ZL_imag,
201                           f_start=1e6, f_end=1e9, n=300, save=True):
202     """Plot VSWR vs. frequency sweep."""
203     freqs, vswrs = sweep_frequency_vswr(R, L, G, C, ZL_real, ZL_imag,
204                                         f_start, f_end, n)
205
206     fig, ax = plt.subplots(figsize=(7, 4))
207     ax.plot(freqs / 1e6, vswrs, color=COLORS["vswr"], linewidth=2)
208     _style_axes(ax,
209                title="VSWR vs. Frequency",
210                xlabel="Frequency (MHz)",
211                ylabel="VSWR (dimensionless)")
212     ax.axhline(y=1.0, linestyle="--", color="black", linewidth=1,
213              label="VSWR = 1 (perfect match)")
214     ax.set_ylim(bottom=0)
215     ax.legend(fontsize=8)
216     fig.tight_layout()
217     if save:
218         _save_fig(fig, "05_VSWR_vs_frequency.png")
219     return fig
220
221
222 # -----
223 # Plot 6: alpha & beta vs frequency
224 # -----
225 def plot_alpha_beta_vs_frequency(R, L, G, C,
226                                 f_start=1e6, f_end=1e9, n=300, save=True):
227     """Plot attenuation (alpha) and phase constant (beta) vs. frequency."""
228     freqs, alphas, betas, _ = sweep_frequency(R, L, G, C, f_start, f_end, n)
229
230     fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(7, 6), sharex=True)
231
232     ax1.plot(freqs / 1e6, alphas, color=COLORS["alpha"], linewidth=2)
233     _style_axes(ax1,
234                title="Attenuation Constant alpha vs. Frequency",
235                xlabel="",
236                ylabel="alpha (Np/m)")
237
238     ax2.plot(freqs / 1e6, betas, color=COLORS["beta"], linewidth=2)
239     _style_axes(ax2,
240                title="Phase Constant beta vs. Frequency",
241                xlabel="Frequency (MHz)",
242                ylabel="beta (rad/m)")
243
244     fig.suptitle("Propagation Parameters vs. Frequency",
245                fontsize=13, fontweight="bold", y=1.01)
246     fig.tight_layout()
247     if save:
248         _save_fig(fig, "06_alpha_beta_vs_frequency.png")
249     return fig
250
251
252 # -----
253 # Plot 7: Standing Wave Envelope
254 # -----
255 def plot_standing_wave(R, L, G, C, f, d_total, ZL_real, ZL_imag,
256                       n=500, save=True):
257     """
258     Plot the standing wave pattern -- voltage envelope (V_max and V_min).
259
260     Envelope:
261         V_max(z) = |V+| * (1 + |Gamma(z)|)    <- constructive

```

```

262     V_min(z) = |V+| * (1 - |Gamma(z)|)    <- destructive
263     """
264     ZL      = complex(ZL_real, ZL_imag)
265     gamma, _, _ = compute_gamma(R, L, G, C, f)
266     Zo      = compute_Zo(R, L, G, C, f)
267     Gamma_L = compute_reflection_coefficient(ZL, Zo)
268
269     z_arr    = np.linspace(0, d_total, n)
270     Gamma_z  = compute_Gamma_along_line(z_arr, Gamma_L, gamma)
271     V_max    = 1 + np.abs(Gamma_z)
272     V_min    = np.abs(1 - np.abs(Gamma_z))
273
274     fig, ax = plt.subplots(figsize=(7, 4))
275     ax.plot(z_arr, V_max, color=COLORS["voltage"], linewidth=2,
276            label=r"$V_{\max}$ (constructive)")
277     ax.plot(z_arr, V_min, color=COLORS["current"], linewidth=2,
278            linestyle="--", label=r"$V_{\min}$ (destructive)")
279     ax.fill_between(z_arr, V_min, V_max, alpha=0.15,
280                    color=COLORS["envelope"], label="Standing wave region")
281     _style_axes(ax,
282                 title="Standing Wave Envelope Along Transmission Line",
283                 xlabel="Distance from Load z (m)",
284                 ylabel="Voltage Envelope (normalized)")
285     ax.legend(fontsize=8)
286     fig.tight_layout()
287     if save:
288         _save_fig(fig, "07_standing_wave_envelope.png")
289     return fig
290
291
292 # -----
293 # Convenience: generate all 7 plots at once
294 # -----
295 def generate_all_plots(R, L, G, C, f, d_total, ZL_real, ZL_imag,
296                       f_start=1e6, f_end=1e9, save=True):
297     """
298     Generate all 7 plots and return a list of (title, Figure) tuples.
299     """
300     figs = []
301
302     figs.append(("Voltage |V(z)|",
303                 plot_voltage(R, L, G, C, f, d_total, ZL_real, ZL_imag,
304                             save=save)))
305     figs.append(("Current |I(z)|",
306                 plot_current(R, L, G, C, f, d_total, ZL_real, ZL_imag,
307                             save=save)))
308     figs.append(("Reflection |Gamma(z)|",
309                 plot_reflection_coefficient(R, L, G, C, f, d_total,
310                                             ZL_real, ZL_imag, save=save)))
311     figs.append(("Zin vs z",
312                 plot_Zin(R, L, G, C, f, d_total, ZL_real, ZL_imag,
313                          save=save)))
314     figs.append(("VSWR vs f",
315                 plot_VSWR_vs_frequency(R, L, G, C, ZL_real, ZL_imag,
316                                         f_start, f_end, save=save)))
317     figs.append(("alpha & beta vs f",
318                 plot_alpha_beta_vs_frequency(R, L, G, C, f_start, f_end,
319                                              save=save)))
320     figs.append(("Standing Wave",
321                 plot_standing_wave(R, L, G, C, f, d_total,
322                                     ZL_real, ZL_imag, save=save)))
323     return figs

```

C. Machine Learning Training and Inference (ml_model.py)

```
1 """
2 ml_model.py
3 =====
4 Machine Learning Model for Transmission Line Parameter Prediction
5 =====
6 Uses per-target Random Forest / Gradient Boosting Regressors with
7 comprehensive physics-informed feature engineering to achieve  $R^2 \geq 0.97$ .
8
9 Strategy:
10 -  $|Z_0|$  and  $\beta$  are nearly analytically deterministic -> RF excels
11 -  $\alpha$  depends on  $R, G, \omega$  -> GB excels with engineered features
12 -  $|\Gamma_L|$  and VSWR depend on ZL vs  $Z_0$  -> add  $Z_0$ -related features
13
14 Training Pipeline:
15 1. Generate 1000 synthetic data points in realistic coaxial/stripline ranges
16 2. First analytically estimate  $Z_0$ , then add it as a feature for  $\Gamma/VSWR$ 
17 3. Log-transform wide-range features and targets
18 4. Train best-performing model per target (RF or GB)
19 5. Evaluate:  $R^2$  score and MAPE on a held-out test set (20%)
20
21 Output targets (5):
22  $|Z_0|$ ,  $\alpha$ ,  $\beta$ ,  $|\Gamma_L|$ , VSWR
23 """
24
25 import numpy as np
26 import os
27 import pickle
28 import warnings
29
30 from sklearn.ensemble import (RandomForestRegressor,
31                               GradientBoostingRegressor)
32 from sklearn.preprocessing import StandardScaler
33 from sklearn.model_selection import train_test_split
34 from sklearn.metrics import r2_score, mean_absolute_percentage_error
35
36 from analytics import (compute_all, compute_gamma, compute_Z0,
37                       compute_reflection_coefficient, compute_VSWR)
38
39 warnings.filterwarnings("ignore")
40
41 # -----
42 # Paths
43 # -----
44 _DIR = os.path.dirname(os.path.abspath(__file__))
45 MODEL_PKL = os.path.join(_DIR, "ml_model.pkl")
46
47 # -----
48 # Realistic coaxial-cable parameter ranges
49 # -----
50 PARAM_RANGES = {
51     "R": (0.01, 150.0), #  $\Omega m/m$ 
52     "L": (50e-9, 600e-9), #  $H/m$ 
53     "G": (1e-6, 1e-2), #  $S/m$ 
54     "C": (10e-12, 300e-12), #  $F/m$ 
55     "f": (1e6, 3e9), #  $Hz$ 
56     "d": (0.05, 20.0), #  $m$ 
57     "|ZL|": (5.0, 500.0), #  $\Omega m$ 
58     "aZL": (-75.0, 75.0), #  $degrees$ 
59 }
60
61 TARGET_NAMES = ["|Z0|", "alpha", "beta", "|Gamma_L|", "VSWR"]
62 FEATURE_NAMES = [
63     "R", "log_L", "log_G", "log_C", "log_f", "d",
```

```

64     "|ZL|", "aZL",
65     "log_omega", "log_wL", "log_wC",
66     "log_|Zs|", "log_|Yp|", "log_Q",
67     "log_Zo_approx",          # approx Zo = sqrt(L/C)
68     "ZL_over_Zo",            # |ZL|/Zo_approx
69     "ZL_real_over_Zo",        # Re(ZL)/Zo_approx
70     "ZL_imag_over_Zo",        # Im(ZL)/Zo_approx
71     "log_alpha_approx",       # approx alpha = R/(2*Zo) + G*Zo/2
72 ]
73
74
75 # -----
76 # Feature engineering (physics-informed)
77 # -----
78 def _engineer_features(R, L, G, C, f, d, ZL_mag, ZL_angle_deg):
79     """Build enriched feature matrix with physics-derived features."""
80     omega = 2 * np.pi * f
81     wL = omega * L
82     wC = omega * C
83     Z_series = np.sqrt(R**2 + wL**2)
84     Y_shunt = np.sqrt(G**2 + wC**2)
85     Q = wL / (R + 1e-15)
86
87     # Approximate Zo from lossless formula (good first-order estimate)
88     Zo_approx = np.sqrt(L / (C + 1e-30))
89
90     # ZL components
91     ZL_real = ZL_mag * np.cos(np.radians(ZL_angle_deg))
92     ZL_imag = ZL_mag * np.sin(np.radians(ZL_angle_deg))
93
94     # ZL vs Zo features
95     ZL_over_Zo = ZL_mag / (Zo_approx + 1e-15)
96     ZLr_over_Zo = ZL_real / (Zo_approx + 1e-15)
97     ZLi_over_Zo = ZL_imag / (Zo_approx + 1e-15)
98
99     # Approximate alpha (low-loss formula)
100     alpha_approx = R / (2 * Zo_approx + 1e-15) + G * Zo_approx / 2
101
102     X = np.column_stack([
103         R,
104         np.log10(np.maximum(L, 1e-15)),
105         np.log10(np.maximum(G, 1e-15)),
106         np.log10(np.maximum(C, 1e-15)),
107         np.log10(np.maximum(f, 1.0)),
108         d,
109         ZL_mag,
110         ZL_angle_deg,
111         np.log10(np.maximum(omega, 1.0)),
112         np.log10(np.maximum(wL, 1e-15)),
113         np.log10(np.maximum(wC, 1e-15)),
114         np.log10(np.maximum(Z_series, 1e-15)),
115         np.log10(np.maximum(Y_shunt, 1e-15)),
116         np.log10(np.maximum(Q, 1e-15)),
117         np.log10(np.maximum(Zo_approx, 1e-15)),
118         ZL_over_Zo,
119         ZLr_over_Zo,
120         ZLi_over_Zo,
121         np.log10(np.maximum(alpha_approx, 1e-15)),
122     ])
123     return X
124
125
126 # -----
127 # Data generation
128 # -----
129 def generate_dataset(n_samples=1000, random_seed=42):

```

```

130     """
131     Generate synthetic dataset by randomly sampling Tx line parameters
132     and computing exact analytical labels.
133
134     Returns
135     -----
136     X          : ndarray (n_samples, 19) -- engineered features
137     y_raw      : ndarray (n_samples, 5)  -- raw targets
138     y_log      : ndarray (n_samples, 5)  -- log/linear transformed targets
139     """
140     rng = np.random.default_rng(random_seed)
141
142     def U(key, n):
143         lo, hi = PARAM_RANGES[key]
144         return rng.uniform(lo, hi, n)
145
146     R          = U("R",      n_samples)
147     L          = U("L",      n_samples)
148     G          = U("G",      n_samples)
149     C          = U("C",      n_samples)
150     f          = U("f",      n_samples)
151     d          = U("d",      n_samples)
152     ZL_mag     = U("|ZL|", n_samples)
153     ZL_angle   = U("aZL", n_samples)
154
155     ZL_real = ZL_mag * np.cos(np.radians(ZL_angle))
156     ZL_imag = ZL_mag * np.sin(np.radians(ZL_angle))
157
158     X = _engineer_features(R, L, G, C, f, d, ZL_mag, ZL_angle)
159
160     y_raw = np.zeros((n_samples, 5))
161     for i in range(n_samples):
162         res = compute_all(R[i], L[i], G[i], C[i], f[i], d[i],
163                          ZL_real[i], ZL_imag[i])
164         y_raw[i, 0] = res["|Zo|"]
165         y_raw[i, 1] = res["alpha"]
166         y_raw[i, 2] = res["beta"]
167         y_raw[i, 3] = res["|Gamma_L|"]
168         y_raw[i, 4] = min(res["VSWR"], 50.0)
169
170     # Log-transform for wide-range targets, keep |Gamma_L| linear
171     y_log = np.column_stack([
172         np.log10(np.maximum(y_raw[:, 0], 1e-15)), # log |Zo|
173         np.log10(np.maximum(y_raw[:, 1], 1e-15)), # log alpha
174         np.log10(np.maximum(y_raw[:, 2], 1e-15)), # log beta
175         y_raw[:, 3],                               # |Gamma_L| linear [0,1]
176         np.log10(np.maximum(y_raw[:, 4], 1.0)),   # log VSWR
177     ])
178
179     return X, y_raw, y_log
180
181
182 # -----
183 # Select best algorithm per target
184 # -----
185 def _best_model_for_target(j):
186     """
187     Return the best sklearn model for each target index.
188     GradientBoosting generalises better on harder, nonlinear targets.
189     """
190     configs = [
191         # |Zo| -- nearly analytical from L, C
192         RandomForestRegressor(
193             n_estimators=600, max_features=0.6,
194             min_samples_leaf=1, n_jobs=-1, random_state=j),
195         # alpha -- sensitive to R, G, omega

```

```

196         GradientBoostingRegressor(
197             n_estimators=500, learning_rate=0.05,
198             max_depth=6, subsample=0.8,
199             min_samples_leaf=2, random_state=j),
200         # beta -- nearly analytical from L, C, omega
201         RandomForestRegressor(
202             n_estimators=600, max_features=0.6,
203             min_samples_leaf=1, n_jobs=-1, random_state=j),
204         # |Gamma_L| -- depends on Zo vs ZL ratio
205         GradientBoostingRegressor(
206             n_estimators=500, learning_rate=0.05,
207             max_depth=6, subsample=0.8,
208             min_samples_leaf=2, random_state=j),
209         # VSWR -- monotonic function of |Gamma_L|
210         GradientBoostingRegressor(
211             n_estimators=500, learning_rate=0.05,
212             max_depth=6, subsample=0.8,
213             min_samples_leaf=2, random_state=j),
214     ]
215     return configs[j]
216
217
218 # -----
219 # Model training
220 # -----
221 def train_model(n_samples=1000, test_size=0.2, random_seed=42):
222     """
223     Train one model per target on synthetic Tx line data.
224
225     Returns
226     -----
227     model_bundle : dict
228     """
229     print(f"[ML] Generating {n_samples} synthetic data points...")
230     X, y_raw, y_log = generate_dataset(n_samples=n_samples,
231                                       random_seed=random_seed)
232
233     X_tr, X_te, \
234     y_log_tr, y_log_te, \
235     y_raw_tr, y_raw_te = train_test_split(
236         X, y_log, y_raw,
237         test_size=test_size, random_state=random_seed
238     )
239
240     scaler = StandardScaler()
241     X_tr_s = scaler.fit_transform(X_tr)
242     X_te_s = scaler.transform(X_te)
243
244     models = []
245     r2_scores = []
246     mape_scores = []
247
248     print("[ML] Training one model per target...")
249     for j, name in enumerate(TARGET_NAMES):
250         model = _best_model_for_target(j)
251         model.fit(X_tr_s, y_log_tr[:, j])
252         models.append(model)
253
254         pred_log = model.predict(X_te_s)
255         r2 = r2_score(y_log_te[:, j], pred_log)
256
257         # MAPE in original space
258         if name in ("|Zo|", "alpha", "beta", "VSWR"):
259             y_orig = 10 ** y_log_te[:, j]
260             p_orig = 10 ** pred_log
261         else: # |Gamma_L|

```

```

262         y_orig = y_log_te[:, j]
263         p_orig = pred_log
264
265         mask = np.abs(y_orig) > 1e-10
266         mape = mean_absolute_percentage_error(
267             y_orig[mask], p_orig[mask]) * 100 if mask.sum() > 0 else 0.0
268
269         r2_scores.append(r2)
270         mape_scores.append(mape)
271
272         algo = "RF" if isinstance(model, RandomForestRegressor) else "GB"
273         print(f" [{j+1}/5] {name:<14} [{algo}] R^2 = {r2:.4f}"
274               f" MAPE = {mape:.2f}%")
275
276     y_pred_all = np.column_stack([m.predict(X_te_s) for m in models])
277     r2_overall = r2_score(y_log_te, y_pred_all)
278
279     print("\n[ML] Training Complete -- Performance on Test Set")
280     print("=" * 55)
281     print(f" {'Target':<14} {'R^2':>8} {'MAPE (%)':>10}")
282     print("-" * 55)
283     for j, name in enumerate(TARGET_NAMES):
284         print(f" {name:<14} {r2_scores[j]:>8.4f} {mape_scores[j]:>10.2f}")
285     print("-" * 55)
286     print(f" {'Overall R^2':<14} {r2_overall:>8.4f}")
287     print("=" * 55)
288
289     if r2_overall >= 0.97:
290         print("[ML] PASS: Target accuracy (R^2 >= 0.97) ACHIEVED.")
291     else:
292         print(f"[ML] INFO: Overall R^2 = {r2_overall:.4f}"
293               f" (target >= 0.97)")
294
295     # Average feature importance
296     fi_list = []
297     for m in models:
298         if hasattr(m, "feature_importances_"):
299             fi_list.append(m.feature_importances_)
300     fi = np.mean(fi_list, axis=0) if fi_list else np.zeros(X.shape[1])
301
302     bundle = {
303         "scaler": scaler,
304         "models": models,
305         "X_test": X_te,
306         "y_log_test": y_log_te,
307         "y_pred_all": y_pred_all,
308         "r2_scores": r2_scores,
309         "mape_scores": mape_scores,
310         "r2_overall": r2_overall,
311         "feature_importances": fi,
312         "feature_names": FEATURE_NAMES,
313         "target_names": TARGET_NAMES,
314     }
315     return bundle
316
317 # -----
318 # Save / Load model
319 # -----
320
321 def save_model(bundle, path=MODEL_PKL):
322     with open(path, "wb") as f:
323         pickle.dump(bundle, f)
324     print(f"[ML] Model saved to: {path}")
325
326
327 def load_model(path=MODEL_PKL):

```

```

328     with open(path, "rb") as f:
329         bundle = pickle.load(f)
330     print(f"[ML] Model loaded from: {path}")
331     return bundle
332
333
334 def get_or_train_model(n_samples=1000, force_retrain=False):
335     """Return trained model bundle. Load from disk or train fresh."""
336     if not force_retrain and os.path.exists(MODEL_PKL):
337         try:
338             return load_model()
339         except Exception:
340             pass
341     bundle = train_model(n_samples=n_samples)
342     save_model(bundle)
343     return bundle
344
345
346 # -----
347 # Prediction interface
348 # -----
349 def predict(bundle, R, L, G, C, f, d, ZL_mag, ZL_angle_deg):
350     """
351     Predict Tx line outputs using the trained per-target models.
352
353     Returns dict with predicted |Zo|, alpha, beta, |Gamma_L|, VSWR.
354     """
355     scaler = bundle["scaler"]
356     models = bundle["models"]
357
358     x = _engineer_features(
359         np.array([R]), np.array([L]), np.array([G]), np.array([C]),
360         np.array([f]), np.array([d]),
361         np.array([ZL_mag]), np.array([ZL_angle_deg])
362     ) # shape (1, 19)
363
364     x_s = scaler.transform(x)
365     preds = [m.predict(x_s)[0] for m in models]
366
367     return {
368         "|Zo|_pred":      10 ** preds[0],
369         "alpha_pred":    10 ** preds[1],
370         "beta_pred":     10 ** preds[2],
371         "|Gamma_L|_pred": np.clip(preds[3], 0.0, 1.0),
372         "VSWR_pred":     max(1.0, 10 ** preds[4]),
373     }
374
375
376 # -----
377 # ML Performance Report Plot
378 # -----
379 def plot_ml_performance(bundle, save=True):
380     """
381     2x3 figure: actual vs predicted for each target + feature importance.
382     """
383     import matplotlib
384     matplotlib.use("Agg")
385     import matplotlib.pyplot as plt
386
387     y_log_te = bundle["y_log_test"]
388     y_pred_all = bundle["y_pred_all"]
389     r2_scores = bundle["r2_scores"]
390     fi = bundle["feature_importances"]
391
392     fig, axes = plt.subplots(2, 3, figsize=(14, 9))
393     axes = axes.flatten()

```

```

394
395 colors = ["#1f77b4", "#ff7f0e", "#2ca02c", "#d62728", "#9467bd"]
396 for j, name in enumerate(TARGET_NAMES):
397     ax = axes[j]
398     ax.scatter(y_log_te[:, j], y_pred_all[:, j],
399               alpha=0.5, s=18, color=colors[j], edgecolors="none")
400     mn = min(y_log_te[:, j].min(), y_pred_all[:, j].min())
401     mx = max(y_log_te[:, j].max(), y_pred_all[:, j].max())
402     ax.plot([mn, mx], [mn, mx], "k--", linewidth=1, label="Ideal")
403     lbl = f"log10({name})" if name not in ("|Gamma_L|",) else name
404     ax.set_title(f"{lbl} (R^2 = {r2_scores[j]:.4f})",
405               fontsize=10, fontweight="bold")
406     ax.set_xlabel("Actual", fontsize=9)
407     ax.set_ylabel("Predicted", fontsize=9)
408     ax.legend(fontsize=7)
409     ax.grid(True, linestyle="--", alpha=0.5)
410     ax.spines["top"].set_visible(False)
411     ax.spines["right"].set_visible(False)
412
413 # Feature importance
414 ax6 = axes[5]
415 fi_arr = np.array(fi)
416 n_top = min(12, len(fi_arr))
417 idx = np.argsort(fi_arr)[-n_top:]
418 fn = bundle["feature_names"]
419 ax6.barh([fn[i] for i in idx], fi_arr[idx],
420          color="#7f7f7f", alpha=0.8)
421 ax6.set_title("Avg. Feature Importances",
422             fontsize=10, fontweight="bold")
423 ax6.set_xlabel("Importance", fontsize=9)
424 ax6.grid(True, axis="x", linestyle="--", alpha=0.5)
425 ax6.spines["top"].set_visible(False)
426 ax6.spines["right"].set_visible(False)
427
428 fig.suptitle(
429     f"ML Model Performance -- RF/GB (per-target)"
430     f" | Overall R^2 = {bundle['r2_overall']:.4f}",
431     fontsize=13, fontweight="bold"
432 )
433 fig.tight_layout()
434
435 if save:
436     from plots import PLOT_DIR
437     path = os.path.join(PLOT_DIR, "08_ml_performance.png")
438     os.makedirs(PLOT_DIR, exist_ok=True)
439     fig.savefig(path, dpi=150, bbox_inches="tight")
440     print(f"[ML] Performance plot saved: {path}")
441
442 return fig
443
444
445 # -----
446 # Stand-alone entry point
447 # -----
448 if __name__ == "__main__":
449     bundle = get_or_train_model(n_samples=1000, force_retrain=True)
450     plot_ml_performance(bundle, save=True)
451     print("\n[ML] Sample prediction:")
452     pred = predict(bundle,
453                   R=10, L=250e-9, G=1e-4, C=100e-12,
454                   f=100e6, d=2.0,
455                   ZL_mag=75, ZL_angle_deg=30)
456     for k, v in pred.items():
457         print(f" {k:<22} = {v:.6g}")

```

D. Tkinter Graphical User Interface (gui.py)

```
1 """
2 gui.py
3 =====
4 Tkinter GUI -- Transmission Line Analysis & Design Window
5 =====
6 A complete interactive interface providing:
7 - Input fields for all Tx line parameters
8 - "Compute Analytics" button -> shows all derived parameters
9 - "Plot Waveforms" button -> renders all 7 plots in a tabbed viewer
10 - "ML Predict" button -> shows ML-predicted parameters vs. exact values
11 - "Train/Reload Model" button -> (re-)trains the Random Forest model
12 - Status bar with progress messages
13
14 Layout:
15 +-----+
16 |  TITLE BAR  |
17 +-----+
18 |  INPUT PANEL  |  RESULTS PANEL  |
19 | (R, L, G, C, f,  | (Zo, gamma, alpha, beta, Gamma, VSWR, Zin)  |
20 | d, ZL_r, ZL_i)  | + ML Predictions  |
21 +-----+
22 |  BUTTON BAR  |
23 +-----+
24 |  PLOT CANVAS (tabbed -- 7 tabs, one per plot)  |
25 +-----+
26 |  STATUS BAR  |
27 +-----+
28 """
29
30 import tkinter as tk
31 from tkinter import ttk, messagebox
32 import threading
33 import numpy as np
34 import os
35
36 import matplotlib
37 matplotlib.use("TkAgg")
38 from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
39 import matplotlib.pyplot as plt
40
41 from analytics import compute_all
42 from plots import generate_all_plots
43 import ml_model as mlm
44
45 # -----
46 # Colour / font constants
47 # -----
48 BG_MAIN      = "#1e1e2e"      # dark background
49 BG_PANEL     = "#2a2a3e"      # slightly lighter panel
50 BG_INPUT     = "#313147"
51 FG_TEXT      = "#cdd6f4"
52 FG_LABEL     = "#89b4fa"
53 FG_VALUE     = "#a6e3a1"
54 FG_ML        = "#f9e2af"
55 FG_TITLE     = "#cba6f7"
56 BTN_BG       = "#45475a"
57 BTN_FG       = "#cdd6f4"
58 BTN_ACTIVE   = "#585b70"
59 ACCENT       = "#89dceb"
60
61 FONT_TITLE   = ("Segoe UI", 14, "bold")
62 FONT_HEADER  = ("Segoe UI", 10, "bold")
63 FONT_LABEL   = ("Segoe UI", 9)
```

```

64 FONT_VALUE = ("Consolas", 9, "bold")
65 FONT_BTN   = ("Segoe UI", 9, "bold")
66 FONT_STATUS = ("Segoe UI", 8)
67
68 # -----
69 # Default parameter values (coaxial-cable-like example)
70 # -----
71 DEFAULTS = {
72     "R":      "5.0",          # Ohm/m
73     "L":      "250e-9",       # H/m
74     "G":      "1e-4",         # S/m
75     "C":      "100e-12",      # F/m
76     "f":      "100e6",        # Hz    (100 MHz)
77     "d":      "2.0",          # m
78     "ZL_r":   "75.0",         # Ohm  (real part)
79     "ZL_i":   "30.0",         # Ohm  (imaginary part)
80     "f_start": "1e6",         # Hz   (freq sweep start)
81     "f_end":  "1e9",         # Hz   (freq sweep end)
82 }
83
84 # -----
85 # Helper: formatted row in results panel
86 # -----
87 def _fmt(val, unit="", is_complex=False):
88     if is_complex:
89         r, i = np.real(val), np.imag(val)
90         sign = "+" if i >= 0 else "-"
91         return f"{r:.4f} {sign} j{abs(i):.4f} {unit}"
92     if isinstance(val, (float, np.floating)):
93         return f"{val:.6g} {unit}"
94     return f"{val} {unit}"
95
96
97 # -----
98 # Main Application Window
99 # -----
100 class TxLineApp(tk.Tk):
101     def __init__(self):
102         super().__init__()
103         self.title("Transmission Line Analyzer -- Engineering Electromagnetics")
104         self.configure(bg=BG_MAIN)
105         self.resizable(True, True)
106         self.geometry("1280x820")
107         self.minsize(1100, 700)
108
109         # Model bundle (loaded lazily)
110         self._bundle = None
111
112         self._build_ui()
113         self.after(200, self._load_model_async)
114
115     # -----
116     # UI construction
117     # -----
118     def _build_ui(self):
119         """Build all UI sections."""
120         self._build_title()
121         self._build_main_frame()
122         self._build_input_panel()
123         self._build_results_panel()
124         self._build_button_bar()
125         self._build_plot_area()
126         self._build_status_bar()
127
128     def _build_title(self):
129         frm = tk.Frame(self, bg=BG_MAIN)

```

```

130     frm.pack(fill="x", padx=16, pady=(12, 0))
131     tk.Label(frm,
132             text="* Transmission Line Analysis & Design Window",
133             font=FONT_TITLE, bg=BG_MAIN, fg=FG_TITLE).pack(side="left")
134     tk.Label(frm,
135             text="Engineering Electromagnetics | Module-1",
136             font=FONT_LABEL, bg=BG_MAIN, fg=FG_TEXT).pack(side="right")
137
138     def _build_main_frame(self):
139         self._main_frm = tk.Frame(self, bg=BG_MAIN)
140         self._main_frm.pack(fill="x", padx=12, pady=6)
141
142     def _build_input_panel(self):
143         """Left panel: parameter input fields."""
144         panel = tk.LabelFrame(self._main_frm, text=" [load] Input Parameters ",
145                               bg=BG_PANEL, fg=FG_LABEL,
146                               font=FONT_HEADER, bd=1, relief="groove",
147                               padx=10, pady=8)
148         panel.pack(side="left", fill="y", padx=(0, 6))
149
150         self._entries = {}
151
152         fields = [
153             ("R (Ohm/m)", "R", "Resistance per unit length"),
154             ("L (H/m)", "L", "Inductance per unit length"),
155             ("G (S/m)", "G", "Conductance per unit length"),
156             ("C (F/m)", "C", "Capacitance per unit length"),
157             ("f (Hz)", "f", "Frequency"),
158             ("d (m)", "d", "Line length / distance from load"),
159             ("ZL_real (Ohm)", "ZL_r", "Real part of load impedance"),
160             ("ZL_imag (Ohm)", "ZL_i", "Imaginary part of load impedance"),
161             ("f_start (Hz)", "f_start", "Frequency sweep start"),
162             ("f_end (Hz)", "f_end", "Frequency sweep end"),
163         ]
164
165         for row_i, (label, key, tip) in enumerate(fields):
166             tk.Label(panel, text=label, bg=BG_PANEL, fg=FG_LABEL,
167                     font=FONT_LABEL, anchor="w", width=18).grid(
168                 row=row_i, column=0, sticky="w", pady=3)
169             var = tk.StringVar(value=DEFAULTS.get(key, ""))
170             ent = tk.Entry(panel, textvariable=var, width=14,
171                            bg=BG_INPUT, fg=FG_TEXT,
172                            insertbackground=FG_TEXT,
173                            font=FONT_VALUE, bd=0, relief="flat")
174             ent.grid(row=row_i, column=1, padx=(6, 0), pady=3)
175             self._entries[key] = var
176
177     def _build_results_panel(self):
178         """Right panel: computed results display."""
179         panel = tk.LabelFrame(self._main_frm, text=" [chart] Computed Parameters ",
180                               bg=BG_PANEL, fg=FG_LABEL,
181                               font=FONT_HEADER, bd=1, relief="groove",
182                               padx=10, pady=8)
183         panel.pack(side="left", fill="both", expand=True)
184
185         # Analytical results
186         tk.Label(panel, text="-- Analytical Results --",
187                 bg=BG_PANEL, fg=ACCENT,
188                 font=FONT_HEADER).grid(row=0, column=0, columnspan=2,
189                                         sticky="w", pady=(0, 4))
190
191         self._result_vars = {}
192         result_fields = [
193             ("Zo", "|Zo|", "Ohm"),
194             ("/_Zo", "angle_Zo_deg", "deg"),
195             ("gamma (alpha + jbeta)", "gamma_str", ""),

```

```

196         ("alpha (attenuation)", "alpha",          "Np/m"),
197         ("beta (phase)",          "beta",          "rad/m"),
198         ("Gamma at load",          "|Gamma_L|",      ""),
199         ("/_Gamma",                "angle_Gamma_deg", " deg"),
200         ("VSWR",                    "VSWR",         ""),
201         ("Zin",                      "|Zin|",         "Ohm"),
202         ("/_Zin",                    "angle_Zin_deg", " deg"),
203     ]
204
205     for row_i, (label, key, unit) in enumerate(result_fields):
206         tk.Label(panel, text=label, bg=BG_PANEL, fg=FG_LABEL,
207                 font=FONT_LABEL, anchor="w", width=22).grid(
208             row=row_i+1, column=0, sticky="w", pady=2)
209         var = tk.StringVar(value="--")
210         tk.Label(panel, textvariable=var, bg=BG_PANEL, fg=FG_VALUE,
211                 font=FONT_VALUE, anchor="w", width=30).grid(
212             row=row_i+1, column=1, sticky="w", padx=6)
213         self._result_vars[key] = var
214
215     # Divider
216     ttk.Separator(panel, orient="horizontal").grid(
217         row=len(result_fields)+2, column=0, columnspan=2,
218         sticky="ew", pady=8)
219
220     # ML results
221     tk.Label(panel, text="-- ML Predictions (Random Forest) --",
222             bg=BG_PANEL, fg=FG_ML,
223             font=FONT_HEADER).grid(
224         row=len(result_fields)+3, column=0, columnspan=2,
225         sticky="w", pady=(0, 4))
226
227     self._ml_vars = {}
228     ml_fields = [
229         ("|Zo| predicted",          "|Zo|_pred",      "Ohm"),
230         ("alpha predicted",          "alpha_pred",    "Np/m"),
231         ("beta predicted",          "beta_pred",    "rad/m"),
232         ("|Gamma| predicted",        "|Gamma_L|_pred", ""),
233         ("VSWR predicted",          "VSWR_pred",     ""),
234         ("Model R^2 (overall)",      "r2_overall",    ""),
235     ]
236
237     base = len(result_fields) + 4
238     for row_i, (label, key, unit) in enumerate(ml_fields):
239         tk.Label(panel, text=label, bg=BG_PANEL, fg=FG_ML,
240                 font=FONT_LABEL, anchor="w", width=22).grid(
241             row=base + row_i, column=0, sticky="w", pady=2)
242         var = tk.StringVar(value="--")
243         tk.Label(panel, textvariable=var, bg=BG_PANEL, fg=FG_ML,
244                 font=FONT_VALUE, anchor="w", width=30).grid(
245             row=base + row_i, column=1, sticky="w", padx=6)
246         self._ml_vars[key] = var
247
248     def _build_button_bar(self):
249         """Action buttons."""
250         bar = tk.Frame(self, bg=BG_MAIN)
251         bar.pack(fill="x", padx=12, pady=4)
252
253         btn_cfg = dict(bg=BTN_BG, fg=BTN_FG, font=FONT_BTN,
254                       activebackground=BTN_ACTIVE, activeforeground=FG_TEXT,
255                       bd=0, padx=16, pady=6, cursor="hand2", relief="flat")
256
257         tk.Button(bar, text="[settings] Compute Analytics",
258                 command=self._compute, **btn_cfg).pack(side="left", padx=4)
259         tk.Button(bar, text="[plot] Plot All Waveforms",
260                 command=self._plot_all, **btn_cfg).pack(side="left", padx=4)
261         tk.Button(bar, text="[AI] ML Predict",

```

```

262         command=self._ml_predict, **btn_cfg).pack(side="left", padx=4)
263 tk.Button(bar, text="[reload] Train / Reload Model",
264         command=self._train_model, **btn_cfg).pack(side="left", padx=4)
265 tk.Button(bar, text="[repeat] Reset Defaults",
266         command=self._reset_defaults, **btn_cfg).pack(side="left", padx=4)
267
268 def _build_plot_area(self):
269     """Tabbed plot canvas area."""
270     self._nb = ttk.Notebook(self)
271     self._nb.pack(fill="both", expand=True, padx=12, pady=(0, 4))
272
273     # Style the notebook
274     style = ttk.Style()
275     style.theme_use("clam")
276     style.configure("TNotebook",
277                     background=BG_MAIN, borderwidth=0)
278     style.configure("TNotebook.Tab",
279                     background=BTN_BG, foreground=FG_TEXT,
280                     font=FONT_LABEL, padding=(8, 4))
281     style.map("TNotebook.Tab",
282             background=[("selected", BG_PANEL)],
283             foreground=[("selected", ACCENT)])
284
285     self._tab_frames = {}
286     self._tab_canvases = {}
287
288     tab_labels = [
289         "Voltage |V|",
290         "Current |I|",
291         "Reflection |Gamma|",
292         "Zin vs z",
293         "VSWR vs f",
294         "alpha & beta vs f",
295         "Standing Wave",
296         "ML Performance",
297     ]
298     for label in tab_labels:
299         frm = tk.Frame(self._nb, bg=BG_PANEL)
300         self._nb.add(frm, text=f" {label} ")
301         self._tab_frames[label] = frm
302
303     # Placeholder labels in each tab
304     for label, frm in self._tab_frames.items():
305         tk.Label(frm, text=f"[ {label} -- click 'Plot All Waveforms' ]",
306                 bg=BG_PANEL, fg=FG_TEXT, font=FONT_LABEL).pack(
307                 expand=True)
308
309 def _build_status_bar(self):
310     self._status_var = tk.StringVar(value="Ready. Enter parameters and click Compute.")
311     bar = tk.Frame(self, bg="#11111b", bd=1, relief="sunken")
312     bar.pack(fill="x", side="bottom")
313     tk.Label(bar, textvariable=self._status_var,
314             bg="#11111b", fg="#6c7086",
315             font=FONT_STATUS, anchor="w", padx=8).pack(fill="x")
316
317 # -----
318 # Utility
319 # -----
320 def _status(self, msg):
321     self._status_var.set(msg)
322     self.update_idletasks()
323
324 def _get_params(self):
325     """Parse all entry fields and return a dict of floats."""
326     try:
327         p = {k: float(v.get()) for k, v in self._entries.items()}

```

```
328         return p
329     except ValueError as e:
330         messagebox.showerror("Input Error",
331                               f"Invalid parameter value:\n{e}")
332         return None
333
334     def _reset_defaults(self):
335         for k, var in self._entries.items():
336             var.set(DEFAULTS.get(k, ""))
337         self._status("Parameters reset to defaults.")
338
339     # -----
340     # Compute Analytics
341     # -----
342     def _compute(self):
343         p = self._get_params()
344         if p is None:
345             return
346         self._status("Computing analytical parameters...")
347         try:
348             res = compute_all(p["R"], p["L"], p["G"], p["C"],
349                              p["f"], p["d"], p["ZL_r"], p["ZL_i"])
350
351             # Format gamma as string
352             a, b = res["alpha"], res["beta"]
353             sign = "+" if b >= 0 else "-"
354             gamma_str = f"{a:.4e} {sign} j{abs(b):.4e}"
355
356             self._result_vars["|Zo|"].set(_fmt(res["|Zo|"], "Ohm"))
357             self._result_vars["angle_Zo_deg"].set(_fmt(res["angle_Zo_deg"], " deg"))
358             self._result_vars["gamma_str"].set(gamma_str)
359             self._result_vars["alpha"].set(_fmt(res["alpha"], "Np/m"))
360             self._result_vars["beta"].set(_fmt(res["beta"], "rad/m"))
361             self._result_vars["|Gamma_L|"].set(_fmt(res["|Gamma_L|"], ""))
362             self._result_vars["angle_Gamma_deg"].set(
363                 _fmt(res["angle_Gamma_deg"], " deg"))
364             self._result_vars["VSWR"].set(_fmt(res["VSWR"], ""))
365             self._result_vars["|Zin|"].set(_fmt(res["|Zin|"], "Ohm"))
366             self._result_vars["angle_Zin_deg"].set(
367                 _fmt(res["angle_Zin_deg"], " deg"))
368
369             self._status("[OK] Analytics computed successfully.")
370         except Exception as e:
371             messagebox.showerror("Computation Error", str(e))
372             self._status("Error during computation.")
373
374     # -----
375     # Plot All Waveforms
376     # -----
377     def _plot_all(self):
378         p = self._get_params()
379         if p is None:
380             return
381         self._status("Generating all 7 waveform plots -- please wait...")
382         self._compute() # refresh analytics first
383
384     def _worker():
385         try:
386             figs = generate_all_plots(
387                 p["R"], p["L"], p["G"], p["C"],
388                 p["f"], p["d"], p["ZL_r"], p["ZL_i"],
389                 f_start=p["f_start"], f_end=p["f_end"],
390                 save=True
391             )
392             self.after(0, lambda: self._render_plots(figs))
393         except Exception as e:
```

```

394         self.after(0, lambda: messagebox.showerror(
395             "Plot Error", str(e)))
396         self.after(0, lambda: self._status("Plot generation failed."))
397
398     threading.Thread(target=_worker, daemon=True).start()
399
400     def _render_plots(self, figs):
401         """Embed matplotlib figures into Notebook tabs."""
402         tab_labels = [
403             "Voltage |V|",
404             "Current |I|",
405             "Reflection |Gamma|",
406             "Zin vs z",
407             "VSWR vs f",
408             "alpha & beta vs f",
409             "Standing Wave",
410         ]
411         for (title, fig), tab_label in zip(figs, tab_labels):
412             frm = self._tab_frames[tab_label]
413             # Clear old widgets
414             for w in frm.wininfo_children():
415                 w.destroy()
416
417             canvas = FigureCanvasTkAgg(fig, master=frm)
418             canvas.draw()
419             canvas.get_tk_widget().pack(fill="both", expand=True)
420             self._tab_c canvases[tab_label] = canvas
421
422         self._status("[OK] All 7 plots generated and saved to plots_output/")
423
424         # Also try ML performance plot if model is ready
425         if self._bundle is not None:
426             try:
427                 fig_ml = mlm.plot_ml_performance(self._bundle, save=True)
428                 frm = self._tab_frames["ML Performance"]
429                 for w in frm.wininfo_children():
430                     w.destroy()
431                 canvas = FigureCanvasTkAgg(fig_ml, master=frm)
432                 canvas.draw()
433                 canvas.get_tk_widget().pack(fill="both", expand=True)
434                 plt.close(fig_ml)
435             except Exception:
436                 pass
437
438         for fig in [f for _, f in figs]:
439             plt.close(fig)
440
441         # -----
442         # ML Predict
443         # -----
444     def _ml_predict(self):
445         if self._bundle is None:
446             messagebox.showinfo("Model Not Ready",
447                 "ML model is still loading. Please wait a moment.")
448             return
449         p = self._get_params()
450         if p is None:
451             return
452         self._status("Running ML prediction...")
453         try:
454             ZL = complex(p["ZL_r"], p["ZL_i"])
455             ZL_mag = abs(ZL)
456             ZL_angle = np.degrees(np.angle(ZL))
457
458             pred = mlm.predict(
459                 self._bundle,

```

```

460         R=p["R"], L=p["L"], G=p["G"], C=p["C"],
461         f=p["f"], d=p["d"],
462         ZL_mag=ZL_mag, ZL_angle_deg=ZL_angle
463     )
464
465     self._ml_vars["|Zo|_pred"].set(
466         f"{pred['|Zo|_pred']:.4f} Ohm")
467     self._ml_vars["alpha_pred"].set(
468         f"{pred['alpha_pred']:.4e} Np/m")
469     self._ml_vars["beta_pred"].set(
470         f"{pred['beta_pred']:.4e} rad/m")
471     self._ml_vars["|Gamma_L|_pred"].set(
472         f"{pred['|Gamma_L|_pred']:.4f}")
473     self._ml_vars["VSWR_pred"].set(
474         f"{pred['VSWR_pred']:.4f}")
475     self._ml_vars["r2_overall"].set(
476         f"{self._bundle['r2_overall']:.4f}")
477
478     self._status("[OK] ML prediction complete.")
479 except Exception as e:
480     messagebox.showerror("ML Error", str(e))
481     self._status("ML prediction failed.")
482
483 # -----
484 # Train / Reload Model
485 # -----
486 def _train_model(self):
487     self._status("Training ML model (500 samples) -- please wait...")
488
489     def _worker():
490         try:
491             bundle = mlm.get_or_train_model(n_samples=1000,
492                                             force_retrain=True)
493
494             self._bundle = bundle
495             r2 = bundle['r2_overall']
496             status = "PASS" if r2 >= 0.97 else "INFO"
497             self.after(0, lambda: self._status(
498                 f"[{status}] Model trained. R^2 = {r2:.4f}"))
499         except Exception as e:
500             self.after(0, lambda: messagebox.showerror(
501                 "Training Error", str(e)))
502             self.after(0, lambda: self._status("Model training failed. "))
503
504     threading.Thread(target=_worker, daemon=True).start()
505
506 def _load_model_async(self):
507     """Silently load (or train) model in background on startup."""
508     def _worker():
509         try:
510             self._bundle = mlm.get_or_train_model(n_samples=1000,
511                                                     force_retrain=False)
512
513             r2 = self._bundle['r2_overall']
514             self.after(0, lambda: self._status(
515                 f"ML model ready. R^2 = {r2:.4f} -- Enter parameters and click
516                 Compute. "))
517         except Exception as e:
518             self.after(0, lambda: self._status(
519                 f"ML model load failed: {e}. Use 'Train/Reload Model' button. "))
520
521     threading.Thread(target=_worker, daemon=True).start()
522
523 if __name__ == "__main__":
524     app = TxLineApp()
525     app.mainloop()

```

E. Application Launcher and CLI Interface (main.py)

```
1 """
2 main.py
3 =====
4 Transmission Line Analyzer -- Entry Point
5 =====
6 Engineering Electromagnetics | Module-1 Assignment
7
8 Usage:
9     python main.py [--cli] [--train] [--plots]
10
11 Flags:
12     (no flags)    Launch the full GUI application
13     --cli         Run a command-line demonstration (no GUI)
14     --train       Force retrain the ML model and exit
15     --plots       Generate all plots for default parameters and exit
16
17 Author   : Student | Engineering Electromagnetics
18 Course   : Transmission Lines -- Module 1
19 """
20
21 import sys
22 import os
23 import argparse
24 import numpy as np
25
26 # Ensure the project directory is in PYTHONPATH
27 sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
28
29
30 # -----
31 # CLI demonstration (no GUI, no Tkinter dependency)
32 # -----
33 def run_cli_demo():
34     """
35     Run a full command-line demonstration:
36     1. Compute all analytical parameters for a sample Tx line
37     2. Train / load the ML model
38     3. Compare analytical vs. ML-predicted values
39     4. Print a summary table
40     """
41     from analytics import compute_all
42     from ml_model import get_or_train_model, predict
43
44     print("\n" + "=" * 60)
45     print("  TRANSMISSION LINE ANALYZER -- CLI Demo")
46     print("  Engineering Electromagnetics | Module-1")
47     print("=" * 60)
48
49     # -- Example parameters -----
50     params = dict(
51         R      = 5.0,          # Ohm/m
52         L      = 250e-9,       # H/m
53         G      = 1e-4,         # S/m
54         C      = 100e-12,      # F/m
55         f      = 100e6,        # Hz   (100 MHz)
56         d      = 2.0,          # m
57         ZL_real = 75.0,         # Ohm  -- must match compute_all() signature
58         ZL_imag = 30.0,        # Ohm
59     )
60
61     print("\n[1] INPUT PARAMETERS")
62     print("-" * 40)
63     for k, v in params.items():
```

```

64         print(f"    {k:<8} = {v:.4g}")
65
66     # -- Analytical computation -----
67     print("\n[2] ANALYTICAL RESULTS")
68     print("-" * 40)
69     res = compute_all(**params)
70
71     a, b = res["alpha"], res["beta"]
72     sign = "+" if b >= 0 else "-"
73     print(f"    {'Zo':<20} = {res['|Zo|']:.4f} Ohm /{res['angle_Zo_deg']:.2f} deg")
74     print(f"    {'gamma':<20} = {a:.4e} {sign} j{abs(b):.4e} 1/m")
75     print(f"    {'alpha':<20} = {res['alpha']:.6e} Np/m")
76     print(f"    {'beta':<20} = {res['beta']:.6e} rad/m")
77     print(f"    {'|Gamma_L|':<20} = {res['|Gamma_L|']:.6f} "
78           f"/{res['angle_Gamma_deg']:.2f} deg")
79     print(f"    {'VSWR':<20} = {res['VSWR']:.4f}")
80     print(f"    {'Zin':<20} = {res['|Zin|']:.4f} Ohm /{res['angle_Zin_deg']:.2f} deg")
81
82     # -- ML model -----
83     print("\n[3] ML MODEL -- Random Forest Regressor")
84     print("-" * 40)
85     bundle = get_or_train_model(n_samples=500, force_retrain=False)
86     print(f"    Overall R^2 (test set) = {bundle['r2_overall']:.4f}")
87     print(f"    Status: {'PASS (>=0.97)' if bundle['r2_overall'] >= 0.97 else 'FAIL (<0.97)'}")
88
89     print("\n    Per-target R^2 scores:")
90     for name, r2, mape in zip(bundle["target_names"],
91                               bundle["r2_scores"],
92                               bundle["mape_scores"]):
93         print(f"        {name:<14}    R^2 = {r2:.4f}    MAPE = {mape:.2f}%")
94
95     # -- Compare analytical vs ML -----
96     print("\n[4] ANALYTICAL vs. ML PREDICTION")
97     print("-" * 60)
98     ZL = complex(params["ZL_real"], params["ZL_imag"])
99     pred = predict(bundle,
100                   R=params["R"], L=params["L"],
101                   G=params["G"], C=params["C"],
102                   f=params["f"], d=params["d"],
103                   ZL_mag=abs(ZL),
104                   ZL_angle_deg=np.degrees(np.angle(ZL)))
105
106     comparisons = [
107         ("|Zo| (Ohm)", res["|Zo|"], pred["|Zo|_pred"]),
108         ("alpha (Np/m)", res["alpha"], pred["alpha_pred"]),
109         ("beta (rad/m)", res["beta"], pred["beta_pred"]),
110         ("|Gamma_L|", res["|Gamma_L|"], pred["|Gamma_L|_pred"]),
111         ("VSWR", res["VSWR"], pred["VSWR_pred"]),
112     ]
113
114     print(f"    {'Parameter':<18} {'Analytical':>14} {'ML Predicted':>14} {'Error%':>9}")
115     print("    " + "-" * 58)
116     for name, exact, ml_val in comparisons:
117         err = abs(exact - ml_val) / (abs(exact) + 1e-12) * 100
118         print(f"    {name:<18} {exact:>14.6g} {ml_val:>14.6g} {err:>8.2f}%")
119
120     # -- Generate plots -----
121     print("\n[5] GENERATING ALL 7 PLOTS (saved to plots_output/)")
122     print("-" * 40)
123     try:
124         from plots import generate_all_plots
125         import matplotlib.pyplot as plt
126         figs = generate_all_plots(
127             params["R"], params["L"], params["G"], params["C"],
128             params["f"], params["d"], params["ZL_real"], params["ZL_imag"],
129             f_start=1e6, f_end=1e9, save=True

```

```

130         )
131         for title, fig in figs:
132             plt.close(fig)
133         from plots import PLOT_DIR
134         print(f" [OK] All plots saved in: {PLOT_DIR}")
135
136         # ML performance plot
137         from ml_model import plot_ml_performance
138         fig_ml = plot_ml_performance(bundle, save=True)
139         plt.close(fig_ml)
140     except Exception as e:
141         print(f" Plot generation error: {e}")
142
143     print("\n" + "=" * 60)
144     print(" CLI Demo Complete.")
145     print("=" * 60 + "\n")
146
147
148 # -----
149 # GUI launcher
150 # -----
151 def run_gui():
152     """Launch the Tkinter GUI application."""
153     try:
154         from gui import TxLineApp
155         app = TxLineApp()
156         app.mainloop()
157     except Exception as e:
158         print(f"[ERROR] GUI failed to start: {e}")
159         print("Try running with --cli flag for command-line mode.")
160         sys.exit(1)
161
162
163 # -----
164 # Entry point
165 # -----
166 def main():
167     parser = argparse.ArgumentParser(
168         description="Transmission Line Analyzer -- Engineering Electromagnetics"
169     )
170     parser.add_argument("--cli", action="store_true",
171                         help="Run CLI demonstration (no GUI)")
172     parser.add_argument("--train", action="store_true",
173                         help="Force retrain ML model and exit")
174     parser.add_argument("--plots", action="store_true",
175                         help="Generate plots for default parameters and exit")
176
177     args = parser.parse_args()
178
179     if args.train:
180         from ml_model import get_or_train_model, plot_ml_performance
181         import matplotlib.pyplot as plt
182         bundle = get_or_train_model(n_samples=500, force_retrain=True)
183         fig = plot_ml_performance(bundle, save=True)
184         plt.close(fig)
185         print("Model training complete. Exiting.")
186         return
187
188     if args.plots:
189         from plots import generate_all_plots
190         import matplotlib.pyplot as plt
191         figs = generate_all_plots(
192             5.0, 250e-9, 1e-4, 100e-12,
193             100e6, 2.0, 75.0, 30.0,
194             f_start=1e6, f_end=1e9, save=True
195         )

```

```
196         for _, fig in figs:
197             plt.close(fig)
198         from plots import PLOT_DIR
199         print(f"Plots saved in: {PLOT_DIR}")
200         return
201
202     if args.cli:
203         run_cli_demo()
204         return
205
206     # Default: launch GUI
207     run_gui()
208
209
210 if __name__ == "__main__":
211     main()
```